Lab0实验
环境搭建

讲解人 课程助教 肖同欢

国家高性能计算中心(合肥)、信息与计算机国家级实验教学示范中心

计算机科学与技术学院

2025年10月09日

目录



- Lab0 实验准备
- Linux 环境配置
- LLVM 等软件环境配置
- C++ 常见语法
- 在 CMake 构建的大型项目中如何 Debug

Lab0 实验准备

本次实验用于指导同学们进行环境的搭建以及带领同学们熟悉实验中常用的 C++ 特性以及 Git 的操作，主要有以下内容：

- 搭建 Ubuntu 22.04 虚拟机环境并下载本课程所需软件包
- 测试环境配置是否成功
- 了解基本的 C++ STL(标准模版库) 使用
- 掌握大型 CMake 项目的 Debug 方法

Linux 环境配置

本学期实验使用的 **核心软件** 版本为：

- Ubuntu 22.04
- LLVM 14.0.0
- Git 2.34.1
- CMake 3.22.1
- Flex 2.6.4
- Bison 3.8.2

注意

如果你已经安装了 WSL，可以直接使用 WSL。

本实验在 WSL 2.0 Ubuntu 22.04.5 上测试通过！

无论是虚拟机还是WSL，都可以通过VSCode的ssh连接来进行实验，所以选择适合自己的方式就好。

LLVM 等软件环境配置

LLVM 是一个用 C++ 编写的开源编译器基础设施项目，包括模块化编译器组件和工具链。它提供了一种称为 LLVM IR 的中间代码，介于高级语言和汇编语言之间，本课程的实验核心围绕 LLVM 展开。

Clang 是 LLVM 编译器工具集的前端，用于编译 C、C++ 和 Objective-C。在配置 LLVM 环境时，需要安装 Clang。

```
# 更新软件源
sudo apt update
# 更新软件（可选）
sudo apt upgrade
# 安装 LLVM & Clang & 调试工具
sudo apt install clang llvm
```

LLVM 等软件环境配置

Git 是一个分布式版本控制软件，目前已经成为最流行的版本管理工具。我们将会经常利用到它的分支合并与分支追踪的功能。

GDB 是一款强大的开发工具，可用于调试各种编程语言的应用程序，帮助开发者诊断和解决代码中的问题。它在开发和调试过程中起着重要作用，有助于提高软件质量和开发效率。

```
sudo apt install git
```

```
sudo apt install gdb
# 查看是否成功安装
gdb --version
```

LLVM 等软件环境配置

在 Ubuntu 中，为了简化安装过程，你可以使用 build-essential 软件包。它集成了编译 C、C++ 等语言所需的关键工具和库，包括编译器、链接器以及其他常用构建工具和库。这样，你就不需要单独安装 gcc、g++、make 等包，让编译和构建软件更加便捷。

```
sudo apt install build-essential

# 查看是否成功安装
gcc --version
```

LLVM 等软件环境配置

CMake 是一个用于管理和生成跨平台构建系统的开源工具。它可以帮助开发人员轻松地配置、构建和测试他们的项目，而无需关心目标操作系统或编译器的细节。CMake 的主要目标是使跨平台开发更加容易和可维护，因此它被广泛用于许多开源和商业项目中。

```
# 安装 cmake
sudo apt install cmake
# 检查 cmake 是否正确安装
cmake --version
```

LLVM 等软件环境配置

Flex 是一个用于生成词法分析器的工具。它简化了词法分析器的创建过程，只需提供正则表达式描述词法规则，Flex 就能自动生成相应的 C 代码。

Bison 是一款解析器生成器，用于将 LALR 文法转换为可编译的 C 代码，从而显著降低了手动设计解析器的工作量。Bison 是对早期 Unix 上的 Yacc 工具的重新实现，因此通常使用 .y 文件扩展名来表示它的语法规则文件（Yacc 是 Yet Another Compiler Compiler 的缩写）。

```
sudo apt-get install flex bison

# 查看是否成功安装
flex --version
bison --version
```

LLVM 等软件环境配置

测试

为了测试配置是否成功，我们提供了一个简单的测试程序 Test.c，其内容如下：

```c
#include <stdio.h>

int Grade[1];
int Degree[1];
int Number[1];
int grade_mul;
int degree_mul;

void StudentNumber(int u[], int v[], int k[]) {
    int grade;
    int degree;
    int number;
    int result;
    grade = u[0];
    degree = v[0];
    number = k[0];
    result = grade * grade_mul + degree * degree_mul + number;
    // 若学号前缀不是 SA 可于此修改
    printf("SA%d\n",result);
    return;
}

/**
 * @brief 修改 main 函数中的信息输出你的学号
 *
 * @param Grade[0]: 你的年级，如 20，21，22 等
 * @param Degree[0]: 你的专业代号，如计科是 11
 * @param Number[0]: 你的学生序号，如 0011（为了避免出现八进制，这个值不能以0为开头），4514 等
 * @return int
 */
int main(void) {
    Grade[0] = 20;
    Degree[0] = 11;
    Number[0] = 4514;
    grade_mul = 1000000;
    degree_mul = 10000;
    StudentNumber(Grade,Degree,Number);
    return 0;
}
```

LLVM 等软件环境配置

它的作用是输出你的学号，请你填写相关信息后，在虚拟机/服务器中执行如下指令运行：

```
# 该指令对 Text.c 进行编译，并生成其对应的 LLVM IR 代码，记为 Test.ll。
$ clang -S -emit-llvm Test.c -o Test.ll
```

随后，我们使用 lli 执行该 IR 文件：

```
$ lli Test.ll && echo $?
SA20114514   # 第一行结果应为你的学号
0            # main 函数返回值，应该为 0
```

C++ 常见语法

- 类与对象
- 继承与多态
- 标准模板库 (STL) 与容器

C++ 常见语法---类与对象

C++ 类是面向对象编程的核心概念之一，可以看作是对 C 语言结构体的扩展。类不仅可以包含数据成员（类似于结构体中的字段），还可以包含函数成员（方法），用于操作这些数据。

```
class MyClass {
private:
    int data_; // 私有成员变量

public:
    // 构造函数
    MyClass(int data) : data_(data) {}

    // 析构函数
    ~MyClass(){}

    // 公有成员函数
    void display() const {
        printf("Data: %d\n", data_);
    }
};
```

C++ 常见语法---类与对象

> C++ 类是**面向对象**编程的核心概念之一，可以看作是对 C 语言结构体的扩展。类不仅可以包含数据成员（类似于结构体中的字段），还可以包含函数成员（方法），用于操作这些数据。

访问控制

- `private`：私有成员只能被类的成员函数访问。
- `public`：公有成员可以在类外部访问。
- `protected`：保护成员可以被继承类访问。

构造函数和析构函数

- **构造函数**：用于初始化对象。名称与类名相同，无返回类型。
- **析构函数**：用于清理对象。以 `~` 开头，名称与类名相同。

析构函数是 C++ “资源获取即初始化”（RAII）的核心回调，它把“手工释放”变成“自动清理”，从根本上杜绝大多数资源泄漏。

C++ 常见语法---继承与多态

继承是面向对象编程中的重要特性，通过继承，子类可以从基类中继承属性和方法，从而实现代码的重用和扩展。此外，通过函数重写和虚函数，C++ 还提供了多态性，允许基类指针或引用调用子类的实现。

```
struct Animal {
  void eat() {
      std::cout << "Eating" << std::endl;
  }
};

struct Cat : public Animal {
  void nyan() {
      std::cout << "Nyan~" << std::endl;
  }
};

struct Dog : public Animal {
  void bark() {
      std::cout << "Bark!" << std::endl;
  }
};
```

C++ 常见语法---继承与多态



> **Struct和Class有什么区别?**

在 C++ 里，`class` 和 `struct` 几乎可以完成彼此的所有功能，**唯一的语法区别**是：

表格 复制

特性	**struct**	**class**
默认成员访问权限	public	private
默认继承方式	public	private

仅此而已。其余能力（构造函数、析构函数、模板、继承、多态、虚函数、友元、运算符重载……）二者完全等价。

C++ 常见语法---继承与多态

函数重写（**Override**）是指子类可以提供一个与基类中同名、参数列表相同的函数，以替换基类中的实现。当基类的函数被声明为**虚函数**（**virtual**）时，子类可以选择重写这个函数，从而实现多态性。

```
struct Animal {
  virtual void say() const {
      std::cout << "Animal sound" << std::endl;
  }
};

struct Cat : public Animal {
  void say() const override {  // 重写基类的 say 函数
      std::cout << "I'm a cat" << std::endl;
  }
};

struct Dog : public Animal {
  void say() const override {  // 重写基类的 say 函数
      std::cout << "I'm a dog" << std::endl;
  }
};
```

C++ 常见语法---继承与多态

虚函数（**virtual**）使得基类的指针或引用在运行时能够调用子类的实现，这是多态性的核心。基类中的函数被声明为虚函数后，当通过基类指针或引用调用这个函数时，实际执行的是子类的重写版本。

```
int main() {
  Animal* a;
  Cat c;
  Dog d;

  a = &c;
  a->say(); // 输出 "I'm a cat"

  a = &d;
  a->say(); // 输出 "I'm a dog"

  return 0;
}
```

C++ 常见语法---标准模板库（STL）与容器



STL (Standard Template Library)，意为标准模板库，包含了许多常用的数据结构，在我们的实验中你可能会用到：

std::vector：动态数组，支持快速随机访问和自动扩展。

std::map：有序关联容器，使用红黑树实现，键值对按键排序。

std::set：有序集合容器，元素唯一且按排序规则排列。

std::unordered_map：无序关联容器，使用哈希表实现，支持快速查找。

std::list：双向链表，适合频繁的插入和删除操作。

编程中经常使用到key/value的形式表示数据之间的关系
STL库提供了map和unordered_map容器，对应处理key/value数据的存储、查找等问题
map内key/value是有序的，unordered_map则是无序的
使用时需要引入<map>头文件

C++ 常见语法---标准模板库 (STL) 与容器

```
map<int, string> node;                          // 定义变量
map<int, string>::iterator iter;                // 定义迭代器 iter
node[16] = "张三";                               // 以数组下标的形式
node.insert(pair<int, string>(28, "李四"));      // 直接插入键值对，pair定义了一个键值对，
                                                // 对应map的key和value。

node[78] = "陆七";
node[58] = "陈二";
node[39] = "王五";

for(iter = node.begin(); iter != node.end(); ++iter) {
        cout<<"工号"<<iter->first<<": "<<iter->second<<", "<<endl;
}
// 输出“工号16：张三，工号28：李四，39：王五，工号58：陈二，工号78：陆七”
```

C++ 常见语法---标准模板库 (STL) 与容器

```
node.erase(58);                                    // 使用key删除key=58的节点

iter = node.find(78);                              // 使用迭代器查找key=78的节点
node.erase(iter);                                  // 删除key=78的节点

node[28] = "赵四";                                 // 仅能修改 value 的值

for(iter = node.begin(); iter != node.end(); ++iter) {
        cout<<"工号"<<iter->first<<": "<<iter->second<<", "<<endl;
}
// 输出“工号16：张三，工号28：赵四，39：王五”
```

C++ 常见语法---标准模板库 (STL) 与容器

```
std::string str = "Hello";
std::cout << str;                          // Hello
std::cout << str.size();                   // 5 (length和size完全相同)
std::cout << str[0];                       // H
std::cout << str.append(" World");         // Hello World
std::cout << str + "!";                    // Hello World!
std::cout << str.substr(6, 5);             // World
str.push_back( '!' );
std::cout << str;                          // Hello World!
```

length: 返回字符串长度

push_back(c): 在字符串末尾添加一个字符 c

append(str): 在字符串末尾添加字符串 str

substr: 取子串

在 CMake 构建的大型项目中如何 Debug

克隆需要的测试工程并构建项目

我们对lab0所用到的项目进行clone，
并生成可执行文件。

```
git clone https://github.com/USTC-Compiler-2025/debugExample
cd debugExample
git checkout lab0
```

```
mkdir build
cd build
cmake ..
make
```

在 CMake 构建的大型项目中如何 Debug

克隆需要的测试工程并构建项目

我们对lab0所用到的项目进行clone，
并生成可执行文件。

```
git clone https://github.com/USTC-Compiler-2025/debugExample
cd debugExample
git checkout lab0
```

```
mkdir build
cd build
cmake ..
make
```

在 CMake 构建的大型项目中如何 Debug



如果出现了错误怎么办?

在Lab0里面，我们的实际输出与预期输出不符，需要通过debug来找出源码的问题。

那应该怎么debug?

```
jyjs@JHost stl_debug % ./build/src/lab0_debug -t
Hello, from stl_debug!
MyMyI'mStudent object created
Student destructor called
Human destructor called
Human destructor called
```

而阅读这个项目的源码我们可以得知，期望的输出是：

```
jyjs@JHost stl_debug % ./build/src/lab0_debug -t
Hello, from stl_debug!
My name is John Doe and I am 25 years old
My name is Jane Doe and I am 20 years old
I'm from MIT
Student object created
Student destructor called
Human destructor called
Human destructor called
```

在 CMake 构建的大型项目中如何 Debug



json 文件设置及 debug 示例

.vscode/launch.json 文件记录了调试器需要的基本信息，它是由 vscode 自动生成的。（如果没有自动生成，也可以自己创建）

```
{
  // 使用 IntelliSense 了解相关属性。
  // 悬停以查看现有属性的描述。
  // 欲了解更多信息，请访问: https://go.microsoft.com/fwlink/?linkid=830387
  "version": "0.2.0",
  "configurations": [
    {
      "type": "lldb",
      "request": "launch",
      "name": "Debug",
      "program": "${workspaceFolder}/<executable file>", //TODO: Add executable fi
      "args": [
        "<arguments>" // TODO: Add arguments here
      ],
      "cwd": "${workspaceFolder}"
    }
  ]
}
```

在 CMake 构建的大型项目中如何 Debug

json 文件设置及 debug 示例

读工程中的 CMakeLists.txt 文件即可得知，生成的可执行文件目录为**${workspaceFolder}/build/src/**，可执行文件名为**lab0_debug**

而 main.cpp:17-20L 要求输入**-t** 参数，因此可以做如下修改（此处修改仅作参考）：

```
...
    "program": "${workspaceFolder}/build/src/lab0_debug",
    "args": [
        "-t",
    ],
...
```

在 CMake 构建的大型项目中如何 Debug

json 文件设置及 debug 示例

在完成上述操作之后，我们就可以进行debug了。

注意：一定要把VSCode打开到拉取的仓库文件夹，否则无法进行正常的debug调试工作。